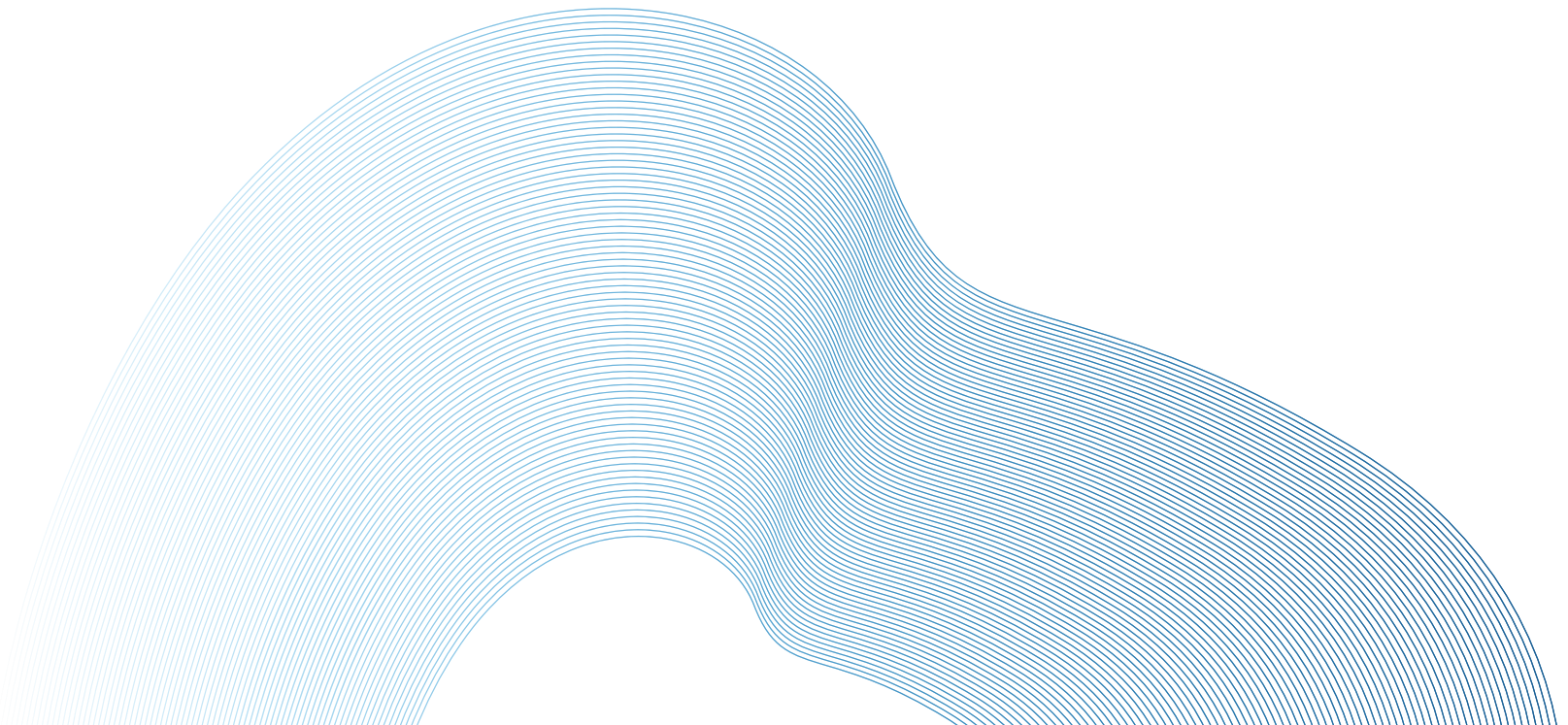
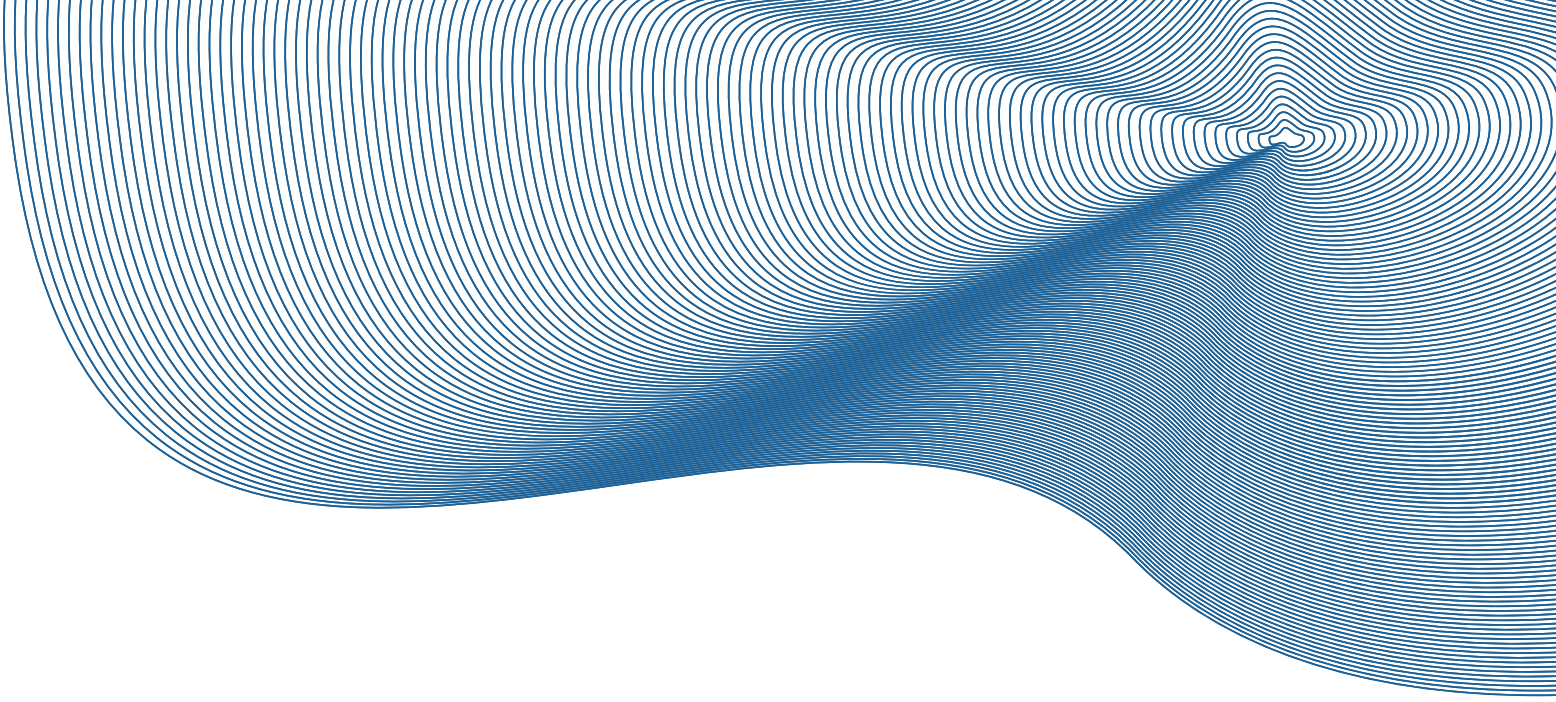




CODE WEB



AULA 09: REVISÃO

ELEMENTOS BÁSICOS

- `<h1>` a `<h6>` – Títulos com hierarquia
- `<p>` – Parágrafo de texto
- `<ul>`, `<ol>`, `<li>` – Listas (não ordenadas / ordenadas)
- `<a href="...">` – Link clicável
- `` – Imagem com descrição alternativa

TABELAS EM HTML

As tabelas em HTML são usadas para organizar dados em linhas e colunas, facilitando a visualização de informações estruturadas, como horários, listas e comparações.

A estrutura básica de uma tabela HTML é feita com a tag `<table>`. Dentro dela, usamos:

- `<tr>` (table row) para criar uma linha;
- `<th>` (table header) para células de cabeçalho, geralmente em negrito;
- `<td>` (table data) para células de dados comuns.

FLEXBOX X GRID

Critério	Flexbox	Grid
Layout	Unidimensional	Bidimensional
Alinhamento	Simples	Complexo
Complexidade	Para layouts simples	Para layouts complexos
Responsividade	Fácil em uma direção	Fácil em várias direções
Uso	Menus, barras laterais	Layouts de página, grades de imagem

INTRODUÇÃO CSS

- Utilizado na estilização de sites.
- seletor {
 propriedade: valor;
}
- **CSS Inline:** Diretamente na tag HTML.
- **CSS Interno (Embedded):** Dentro da tag <style> em <head>.
- **CSS Externo (Linked):** arquivo separado linkado no HTML por <link>.
 <link rel="stylesheet" href="index.css">

SELETORES

- **Seletor de elemento (tag):** Tipo específico de tag.
- **Seletor de classe (.):** Por classe, independente de tipo.
- **Seletor de ID (#):** Elemento com ID exclusiva.
- **Seletor universal (*):** Todos os elementos.
- **Seletor de agrupamento (,):** Seleção múltipla.
- **Seletor de descendentes ():** “Filhos” de outros elementos.

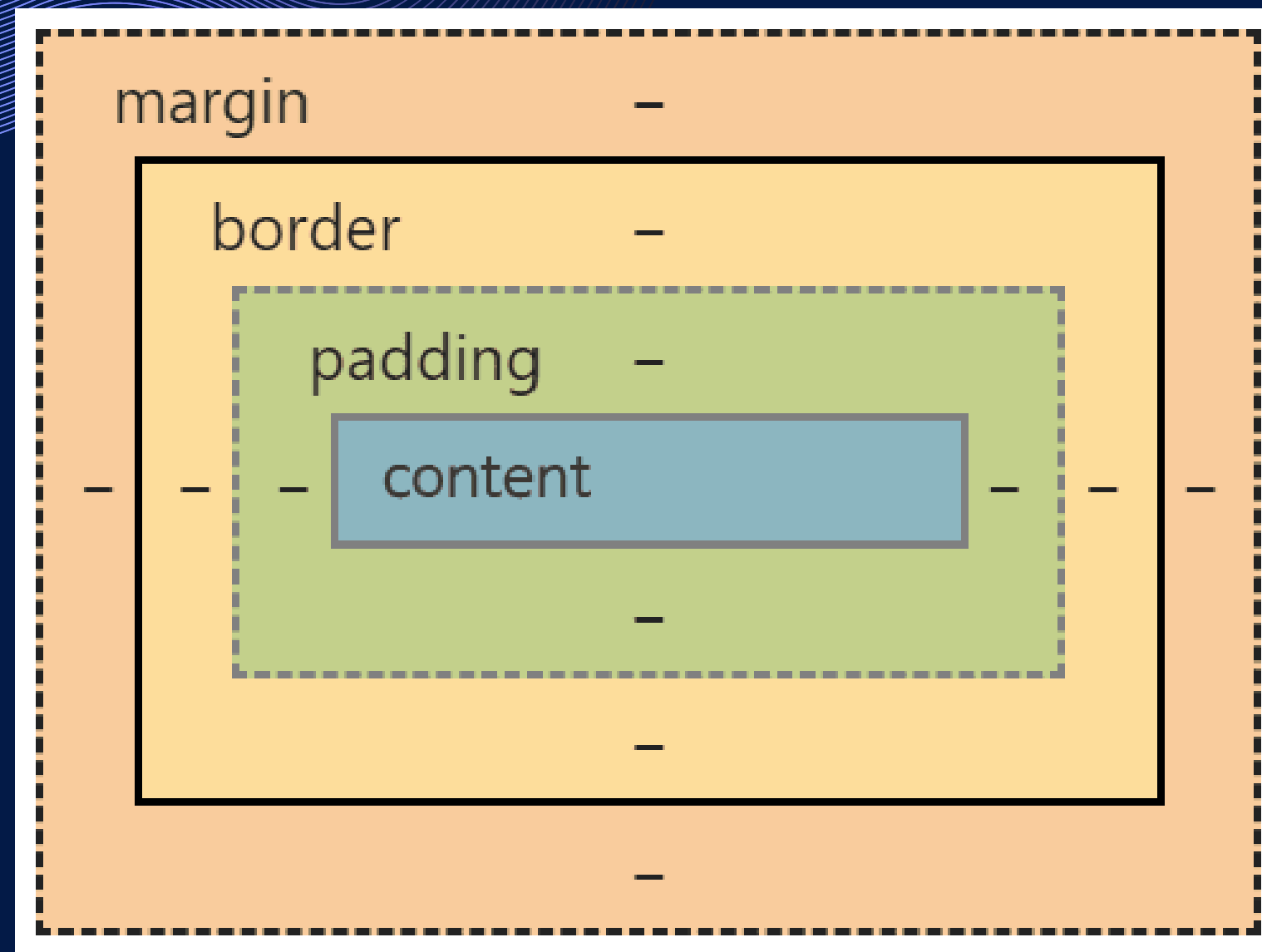
VARIÁVEIS

```
:root{
  --cor-principal: green;
  --tamanho-padrao: 200px;
  --tamanho-secundario: 100px;
}

.classe1{
  color: var(--cor-principal);
  font-size: var(--tamanho-padrao);
}

.classe2{
  color: var(--cor-principal);
  font-size: var(--tamanho-secundario);
}
```


MODELO DE CAIXA (BOX MODEL)



Inspecionar elemento (Developer Tools)

INTRODUÇÃO AO JAVA SCRIPT

- Adicionar a tag `<script>` no fim da tag `<body>`, linkando com " src " o nome do arquivo com fim .js que será utilizado.
 - `<script src = "main.js"></script>`
- Utilizamos o `console.log` para impressão de textos, valores, etc.
- Aba **Console** do **Developer Tools**.

INTRODUÇÃO JAVASCRIPT

- Não é necessário especificar o tipo da variável (dinâmico).
- É permitido assignar uma variável de um tipo para outro tipo.
- **var**, **let** (priorizado) e **const**.
- Dados primitivos: únicos e imutáveis.
- Dados de referência (não primitivos): mutáveis.

DADOS PRIMITIVOS

- **String:** Sequências de caracteres, declaradas por ' ' ou " ", são imutáveis
- **Number:** Representa valores reais ou inteiros.
- **BigInt:** Valores maior que o suportador por Number.
- **Boolean:** Verdadeira ou falso.
- **Null:** Ausência intencional de um valor.
- **Undefined:** Variável declarada mas sem um valor definido.
- **NaN:** Representa um número inválido.

DADOS DE REFERÊNCIA

- **Object:** Coleção de propriedades em pares onde cada uma é definida por meio da propriedade **key-value**.
 - **Dot Notation(.)** e **Array-like Notation([])**.
- **Function:** Objetos que podem ser chamados para executar blocos de códigos.

```
function NomeFuncao(paramet1, paramet2) {  
    // corpo função  
}
```

Utilizar o keyword **return** para que a função retorne o valor de uma variável.

OPERADORES

+ : Adição de valores ou concatenação de strings.

- No caso de **string + número**, o número é convertido para string e então concatenado. Logo `'10' + 20 = 1020`

- : Subtração.

***** : Multiplicação.

/ : Divisão.

%: Resto após realizar uma divisão entre dois valores, dividendo % divisor.

Conversão dos valores da string para números.

OPERADORES

= : Atribuição, permite utilizar **operadores unários**:

- postfix: `x--` e `x++`
- prefix: `--x` e `++x`
- Conversão para positivo (`+x`) e negativo (`-x`)

Utilização da própria variável: `+=`, `-=`, `*=`, `/=`, `%=` e `**=`

- Atribuição de comparação: `>`, `<`, `>=`, `<=`, `==`, `!=`
- `===` e `!==` : estritamente igual (ou diferente), sem conversão automática de valores.

Em strings cada carácter é comparado em valor ASCII. O mesmo ocorre para números em forma de string.

CONDIÇÕES

- **if, else e else if**
 - Operador ternário é a versão reduzida.
 - condição ? seTrueExecutaIsso : seFalseExecutaEste;
- **switch case**
- **comma operator:** Operações da esquerda para direita, retorna o valor da direita.

LAÇOS DE REPETIÇÃO

- **while**
- **for loop**
- **break:** Termina o loop de um laço de repetição.
- **continue:** Termina apenas a execução da iteração atual e não do loop.

Iteração de strings em uma array:

```
let nomes = ['João', 'Gabriel', 'Maria', 'Zezinho'];
let i = 0;
while (i < nomes.length) {
    // nao colocar mais de dois valores dentro de um ${}
    console.log(`${i + 1}.${nomes[i]}*`);
    i++;
}
```


INTRODUÇÃO AO DOM

O DOM (Document Object Model) representa a estrutura do HTML como um objeto JavaScript.

Permite que o JS acesse e modifique elementos da página.

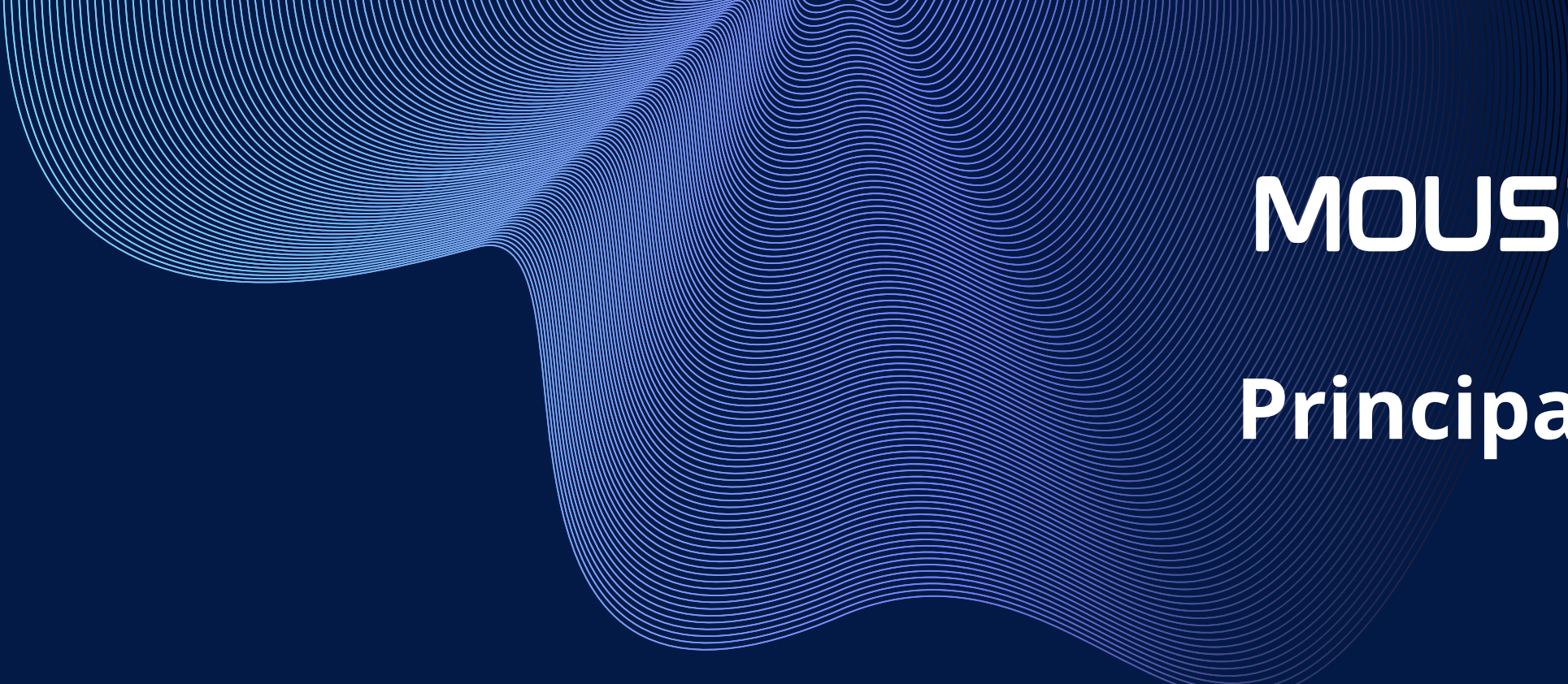
DE QUAIS FORMAS?

Três principais

- `querySelector`
- `addEventListener`
- `getElementById`

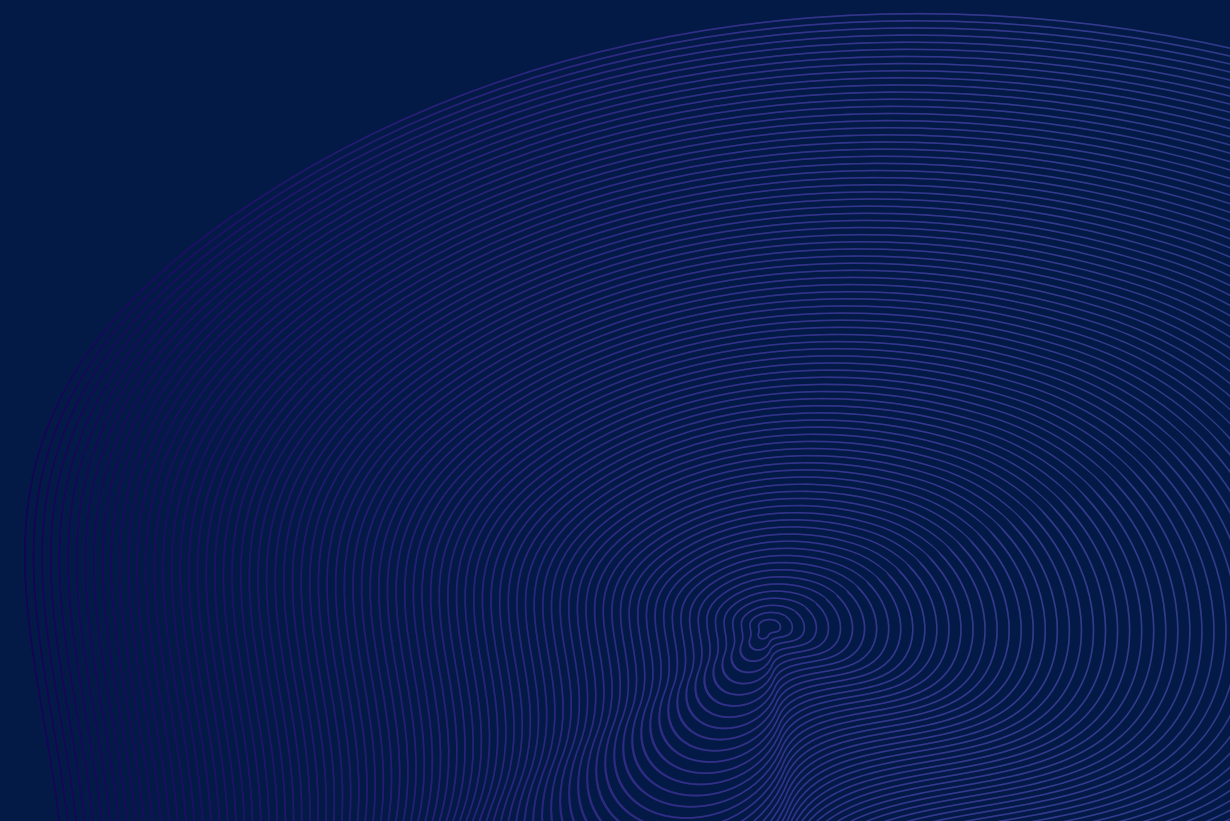


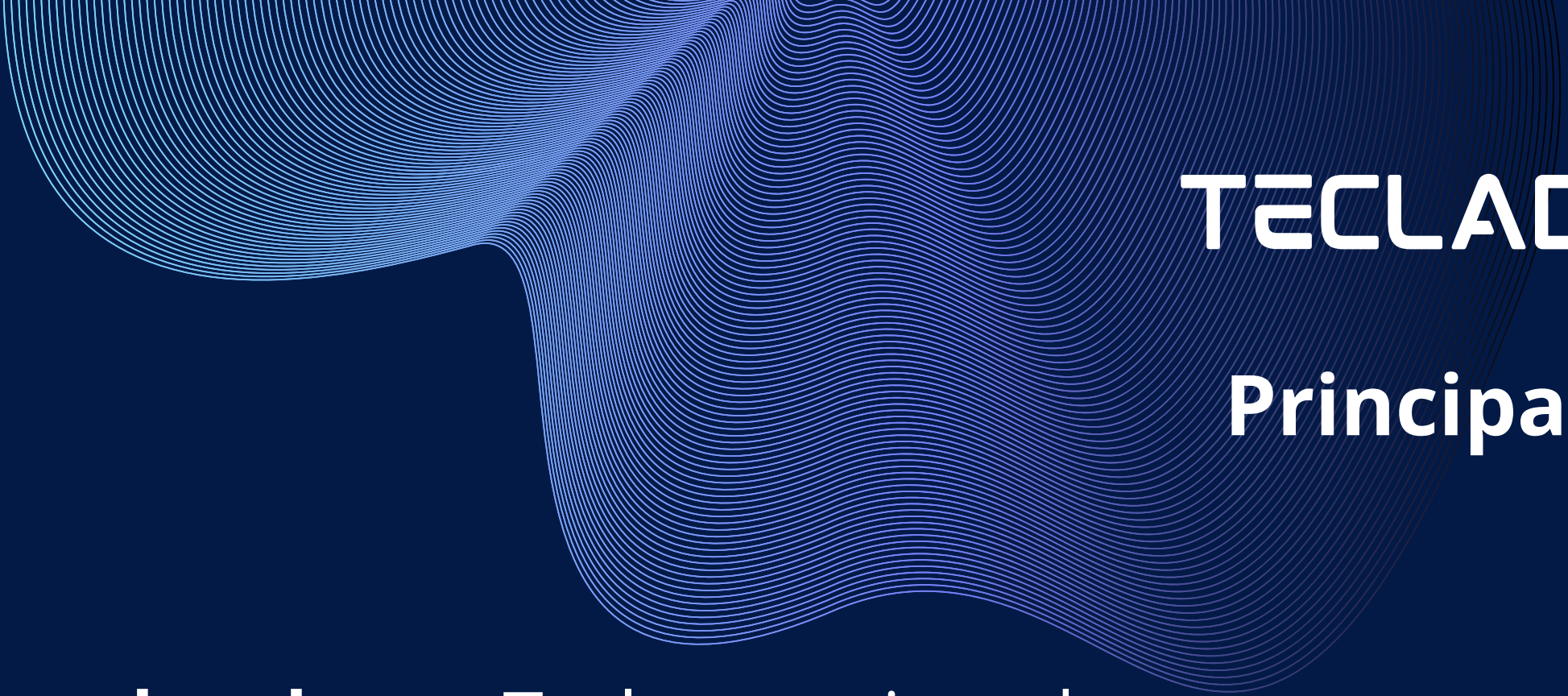
EVENTOS



MOUSE

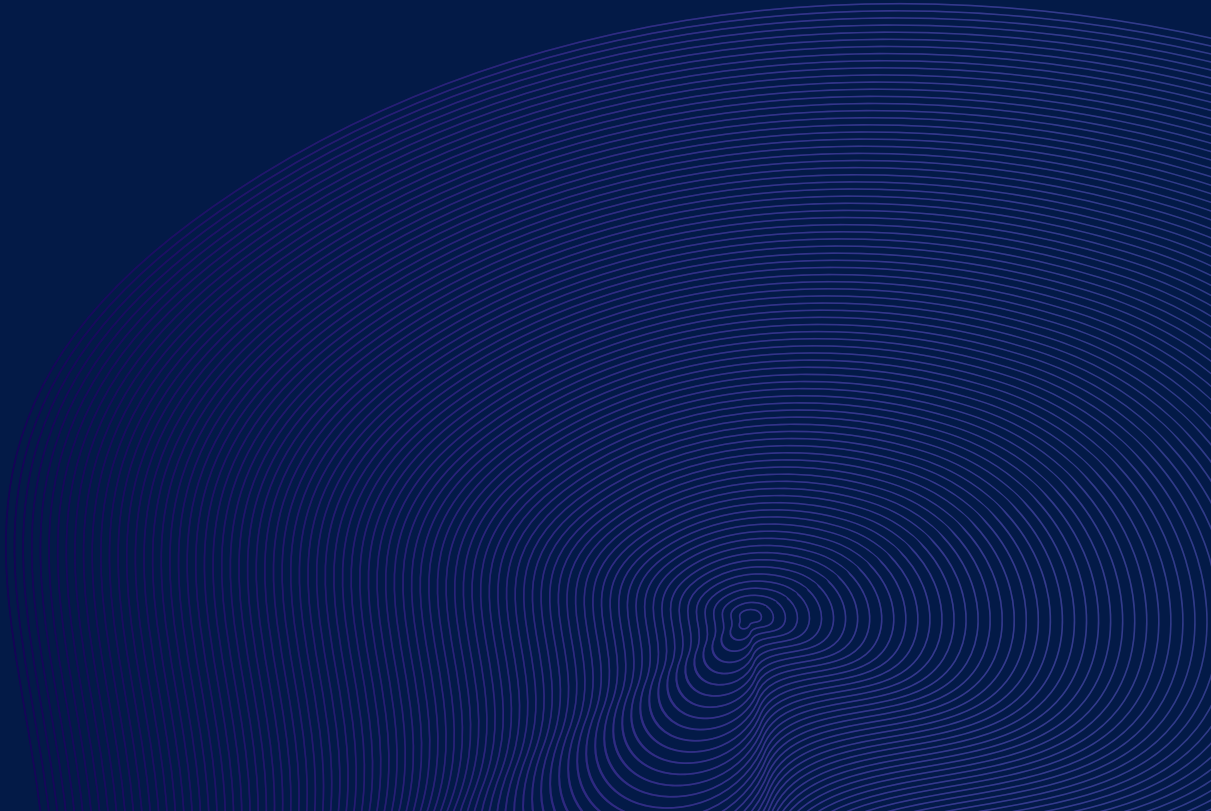
Principais

- **click:** Clique com o botão esquerdo do mouse.
 - **mouseover:** Mouse entra no elemento.
 - **mouseout:** Mouse sai do elemento.
- 



TECLADO

Principais

- **keydown:** Tecla pressionada.
 - **keyup:** Tecla liberada.
- 

FORMULÁRIO

Principais

- **submit:** Envio de formulário.
- **change:** Valor de input/select/textarea alterado.
- **input:** Valor digitado (em tempo real).



OBRIGADO!

That's all Folks!

